APPLIED STATISTICS

Claudio Tessa

Last updated: March 27, 2026

Exam rules

Written exam (30 points) + optional group project (3-6 people) with no deliverables, only a final presentation (6 points).

Contents

1 Introduction **2**
1.1 Linear regression . . . 2
1.2 Multivariate data . . . 2
1.2.1 Supervised learning . . . 3
1.2.2 Unsupervised learning . . . 4
1.3 The prediction problem . . . 4
1.3.1 Bias and Variance . . . 5
1.3.2 Bias-variance trade-off . . . 5

2 Dimensionality reduction **5**
2.1 The curse of dimensionality . . . 5
2.2 Principal Component Analysis (PCA) . . . 7
2.2.1 Information is variability . . . 7
2.2.2 Geometric perspective . . . 8
2.2.3 Finding the new reference system . . . 8
2.2.4 Solving the optimization problem . . . 9
2.2.5 Variance explained . . . 10
2.3 Non-linear dimensionality reduction . . . 11

1 Introduction

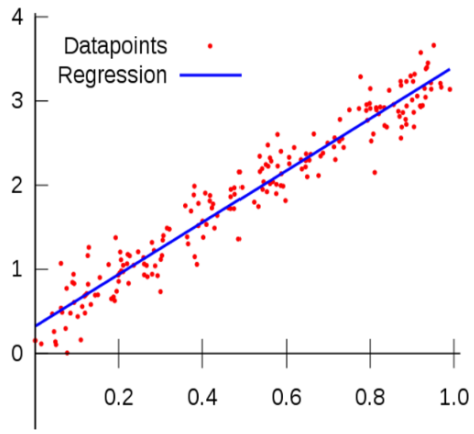
Statistics is concerned with scientific methods for collecting, organising, summarising, presenting and analysing data as well as deriving valid conclusions and making reasonable decisions on the basis of this analysis.

The major difference between machine learning and statistics is their purpose. *Machine learning* models are designed to make the most accurate predictions possible. *Statistical models* are designed for inference about the relationships between variables.

1.1 Linear regression

Linear regression be regarded both as a statistical model and as a machine learning model:

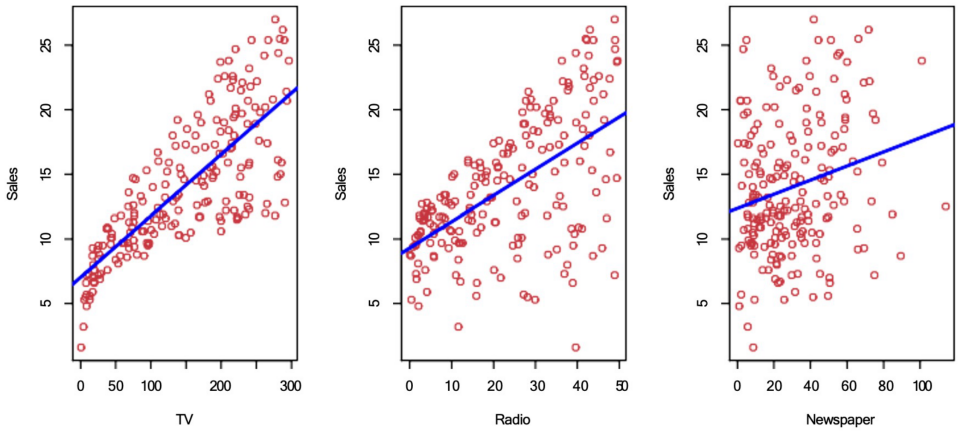
- **Statistical model**: it finds a line that minimizes the mean squared error over all data, having the primary goal to characterize the relationship between the data (covariates) and the output (dependent variable). The model is evaluated through error analysis, confidence instervals, significance tests, etc.
- **Machine learning**: a linear regression can be trained to achieve the best possible prediction. The training is performed on the *training set*, to maximize the accuracy on the *test set*. The model is evaluated based on the accuracy obtained on the test set.



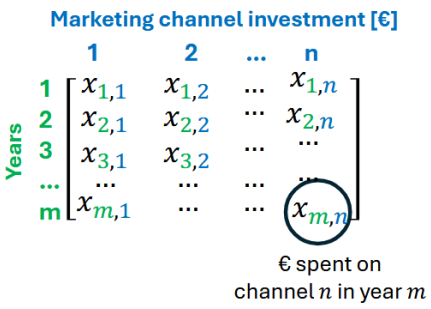
1.2 Multivariate data

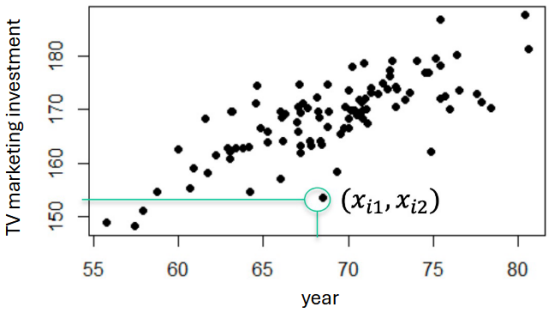
When analyzing complex systems, we work with multivaraite data, which can be represented as a matrix $\mathbb{X} = [x_{ij}] \in \mathbb{R}^{m \times n}$.

Consider the following example: *quantity of products sold vs yearly marketing expenditures on each channel.*



Here, the matrix $\mathbb{X}$ can be represented as





where the i-th row represents a set of coordinates in a multi-dimensional space.

1.2.1 Supervised learning

In the data, it is often possible to identify a specific target variable that we want to analyze or predict. This scenario is known as a **supervised learning** problem. In our example:

- *sales* is the **target** (or response) we want to predict, generally referred to as y;
- *TV*, *radio*, *newspaper* are the **features** (or input, or predictors), named x_1, x_2, x_3.

The relationship between the features and the target is modelled mathematically as $y = f(X) + \varepsilon$, where $X = (x_1, x_2, x_3)$. If the target Y is a continuous number, the task is defined as a **regression problem**, otherwise if Y a categortical value, the task becomes a **calssification problem**.

The ultimate goal of finding a good function f is twofold. First, it allows us to make accurate predictions of Y at new, unseen points where $X = x$. Second, it helps us understand the underlying mechanics of the data: we can identify which comnponents of X are crucial in explaining Y, and which are irrelevant, and how each feature impacts the target.

1.2.2 Unsupervised learning

Not all data, however, comes with a predefined target variable. **Unsupervised learning** operates on **unlabeled data**. Because there is no target Y to predict, the algorithm's primary task is exploratory, analyzing the matrix $\mathbb{X}$ to discover inherent structures, hidden patterns, or relationships on its own. Conceptually, the model attempts to map the data back to itself, represented as $f(X) \approx X$.

1.3 The prediction problem

When building learning systems, the main task is making accurate predictions about unseen data. Consider the following example: *what prediction should we make if we're asked to predict the height of the next student entering the classroom, to minimize the squared prediction error?*

The **Mean Squared Error (MSE)** measures the average of the squares of out errors, represented mathematically as

$$\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

where $\hat{y}_i$ is the predicted target variable, y_i is the true target variable, and n is the number of samples.

If we knew all the heights of all students in the full population, the optimal prediction would simply be the population mean, denoted as μ.

Even in this ideal scenario, our prediction error would not be zero, it would be equivalent to the variance of the through height Y of the next student entering the classroom:

$$E_Y\left[(Y-\mu)^2\right] = Var(Y) = \sigma^2$$

this is known are **irreducible error**, we can't do better than this with any other prediction.

In reality, we don't even have access to the full population or the exact value of μ. To understand how our model behaves under uncertainty, we can imagine a though experiment: drawing a sample of 10 students from the population repeatedly. Each time we resample, we create a new "parallel universe". In each universe, we collect a new sample of heights: $\boldsymbol{Y} = (Y_1, Y_2, \ldots, Y_{10})$, and we use this sample to make a prediction, e.g., the sample mean $\bar{\boldsymbol{Y}}$. By observing the distribution of these predictions across all "parallel universes", we generate a sampling distribution.

By analyzing the average MSE of these predictions, we can decompose the error to understand its origins. The average MSE of these predictions is:

$$AverageMSE = E_{Y,\boldsymbol{Y}}\left[\left(\hat{Y} - Y\right)^2\right]$$

There are two key sources beyond the irreducible error: **bias** and **variance**.

1.3.1 Bias and Variance

Bias is the difference between the mean of our sampling distribution and the actual population mean μ, defined as

$$Bias = E_{\boldsymbol{Y}}[\bar{Y}] - \mu$$

If our model's predictions are systematically too high or too low (on average across all "parallel universes'), the model exhibits bias.

The **variance** measures how much our prediction (i.e., the sample mean) fluctuates from one universe to the next, defined as

$$Var_{\boldsymbol{Y}}(\bar{Y}) = E_{\boldsymbol{Y}}\left[\left(\bar{Y} - E_{\boldsymbol{Y}}[\bar{Y}]\right)^2\right]$$

If our predictions are highly sensitive to the specifice training sample drawn, the model has high variance.

1.3.2 Bias-variance trade-off

Suppose we fit a model $\hat{f}(X)$ to some training data and evaluate it on a new test observation (x_0, y_0). If the true model is $Y = f(X) + \epsilon$ (with $f(X) = E(Y|X = x)$), then the expected prediction error can be explicitly broken down into three components:

$$E[y_0 - \hat{f}(x_0)]^2 = \underbrace{Var[\hat{f}(x_0)]}_{\text{variance}} + \underbrace{Bias[\hat{f}(x_0)]^2}_{\text{bias}} + \underbrace{Var[\varepsilon]}_{\text{noise}}$$

where $Bias[\hat{f}(x_0)] = E[\hat{f}(x_0)] - f(x_0)$.

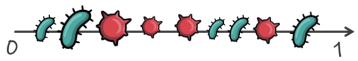
- **Variance**: captures how much the model changes if you train it on a different training set. High variance indicates the model is *over-specialized* or *over-fitting* to its specific dataset.
- **Bias**: what is the inherent error that you obtain from your model, even with infinite training data. This is due to the model being *biased* to a particular kind of solution.
- **Noise**: measures ambiguity due to the data distribution and feature representation. You can never beat this, it is an aspect of the data.

2 Dimensionality reduction

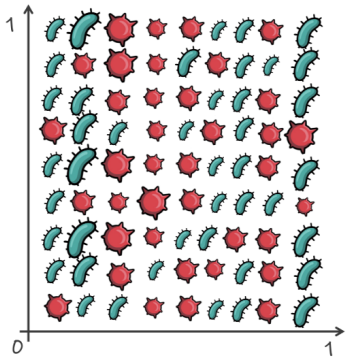
2.1 The curse of dimensionality

In applied mathematics, *curse of dimensionality* refers to the problem caused by the exponential increase in volume associated with adding extra dimensions to a mathematical space.

Imagine we have a single measure between 0 and 1 about two types of bacteria.

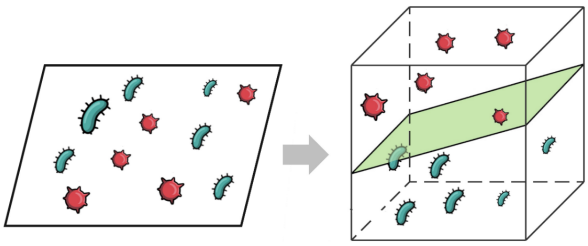


We would like to separate the two groups better, so we add information about another measure (i.e., **we add another dimension**). To maintain that exact same density (in our case, 9 samples per unit length) in a 2D space, we would need 81 samples.



Because we usually only have our initial 9 samples, adding dimensions means the new space we have defined looks much wider and emptier.

At first glance, this extra space might seem like a benefit because wider space makes it easier to separate observations. In our example, the two classes of bacteria cannot be separated by a simple linear model in 2D, but adding just the third dimension does the trick.



More dimensions give our data a higher change of being different (unique), which makes it more sparse. This introduces a higher risk of **overfitting**, meaning that whatever you learn from the data may be extremely specific to that sample.

The curse of dimensionality breaks our ability to interpret data mathematically. A fundamental rule is that *n points always define a linear subspace of at most n-1 dimensions.*

When the number of predictors p we measure for each data point is bigger than the number of data points ($n < p$), those predictors have to be **collinear** or **multicollinear** (the "equivalent" of linearly independent vectors). It looks like we have p different predictor variables, but really some of them are linear combinations of the others, so they don't add any information. This ruins our analysis because our research objective is oftentimes to find out how each predictor is impacting the target variable individually.

Let us now consider the following linear model:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \varepsilon$$

In a healthy model, the parameter β_1 represents the increase in y for a unit increase in x_1, keeping x_2 and (all other covariates) fixed. But in a high-dimensional situation where

multicollinearity exists, our independent variables are highly correlated: if we change x_1, then x_2 also changes, making it impossible to see their individual effect on y.

Lastly, another facet of the course of dimensionality is **distance concentration**. As the dimensionality of the data increases, *all the pairwise distances between different points in the space converge to the same value*. This is counterintuitive: while with more dimensions the data points become more sparse (distant from each other), the gap between the minimum distance and the maximum distance between any set of points become negligible.

If every data point is practically the exact same distance away from every other point, the entire concept of proximity breaks. This avoids any algorithm that relies on measuring distance to determine similarity.

2.2 Principal Component Analysis (PCA)

Consider a dataset represented by a data matrix X of dimensions $n \times p$, where n is the number of statistical units (our observations) and p is the dimension of a single unit (the features or variables we measured). Often, these p variables are correlated.

Principal Component Analysis (PCA) is a dimensionality reduction technique. The primary goal of dimensionality reduction is to summarize this data containing many (p) variables into a smaller set of (k) derived variables. This will transform our original $n \times p$ matrix X into a smaller $n \times k$ matrix M.

However, whenever we reduce dimensions, we leave behind some information from X that is not retained in M. This lost information is called **residual variation**. We must balance the clarity of representation and ease of understanding against the risk of oversimplification (losing important information).

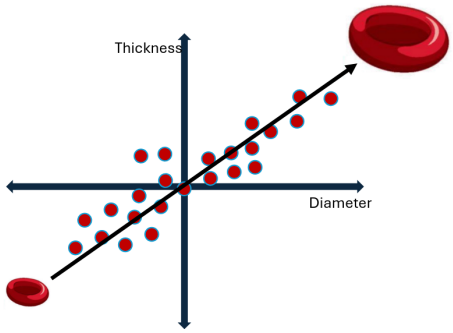
PCA aims at explaining the variability of the features contained in X trhough a few linear combinations of these variables.

2.2.1 Information is variability

Variability is a fundamental concept in statistics. Imagine our statistical units are a cluster of red blood cells, and we have measured two features for each cell: its *thickness* and its *diameter*.

What if we wanted to simplify our dataset and describe these cells using only a single feature instead of two? If we discard thickness and only use the cell's diameter, we lose a lot of information. Cells that have vastly different thicknesses might have the same diameter, making them look much more similar than they actually are. The situation is similar if we decide to keep only the thickness instead.

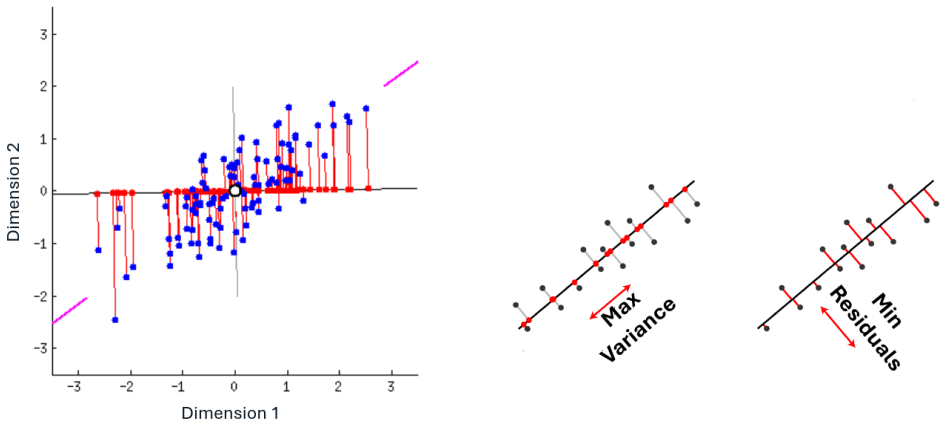
Because information is directly tied to how much the data varies, our goal is to find a way to capture the maximal variability within our dataset. In our example, instead of being forced to choose between the *thickness*-axis or the *diameter*-axis, we can use as axis the direction along which the data points **spread** the most. This new axis is called **principal component**.



2.2.2 Geometric perspective

PCA achieves dimensionality reduction by projecting the data along these directions of maximal variability. When we draw a new line (a principal component) through our data points, our geometric goal is twofold:

1. **Maximize variance**: we want the projected points (the red dots on the new line) to be as spread out as possible.
2. **Maximize residuals**: we want the original data points (the blue dots) to be as physically close to the new line as possible.



The result of PCA is an entirely new reference system for our cloud of points. However, before we can find this new system, we *must* **center** our original data on the origin $(0, 0)$ of the initial reference system. Mathematically, we achieve this by subtracting the mean vector from all points:

$$\mathbf{X}_{centered} = \mathbf{X} - \bar{\mathbf{X}} = (X_1 - \bar{X}_1) + (X_2 - \bar{X}_2)$$

Most of the times, data is also **scaled** by dividing by the standard deviation.

2.2.3 Finding the new reference system

Let's look at the math behind finding this new reference system. Assume we have *centered* data points $x^{(1)}, \ldots, x^{(n)} \in \mathbb{R}^p$, meaning the mean of the points is zero:

$$\frac{1}{n} \sum_{i=1}^{n} x^{(i)} = 0$$

We want to choose a **direction** defined by a unit vector $u \in \mathbb{R}^p$ (i.e., $\|u\|_2 = 1$). If we project a data point $x^{(i)}$ onto this vector u, the resulting 1D score is calculated with:

$$z_i = u^\top x^{(i)} \in \mathbb{R}$$

Since our data is centered, our projected scores z_i will also have a mean of 0. Therefore, the sample variance of our new scores is simply the average of the squared projections:

$$Var(z) = \frac{1}{n}\sum_{i=1}^{n} z_i^2 = \frac{1}{n}\sum_{i=1}^{n} \left(u^\top x^{(i)}\right)^2$$

So our goal is to find:

$$\boxed{\max_{u\in\mathbb{R}^p} \frac{1}{n}\sum_{i=i}^{n} \left(u^\top x^{(i)}\right)^2 \quad \text{s.t.} \quad \|u\|_2 = 1}$$

We can simplify this optimization problem using the empirical covariance matrix, denoted as Σ and defined as:

$$\Sigma = \frac{1}{n}\sum_{i=1}^{n} x^{(i)} x^{(i)\top} \in \mathbb{R}^{p\times p}$$

Bu substituting this into our variance equation, we get:

$$Var(z) = \frac{1}{n}\sum_{i=1}^{n} \left(u^\top x^{(i)}\right)^2 = u^\top \left(\frac{1}{n}\sum_{i=1}^{n} x^{(i)} x^{(i)\top}\right) u = u^\top \Sigma u$$

So the optimization problem becomes:

$$\boxed{\max_{u\in\mathbb{R}^p} u^\top \Sigma u \quad \text{s.t.} \quad \|u\|_2 = 1}$$

2.2.4 Solving the optimization problem

To solve our constraint optimization problem, we can follow these steps:

1. **Turn the constraint into an equation**:

$$\|u\|_2 = 1 \iff u^\top u = 1$$

2. **Build the Lagrangian $\mathcal{L}$**. We introduce a scalar multiplier λ, then combine our objective function and our constraint into a single equation:

$$\mathcal{L}(u, \lambda) = u^\top \Sigma u - \lambda(u^\top u - 1)$$

3. **Take the derivative with respect to u and set it to zero**. To find the maximum, we use calculus. We have that for a symmetric matrix Σ:

$$\nabla_u(u^\top \Sigma u) = 2\Sigma u, \qquad \nabla_u(u^\top u) = 2u$$

   Applying these rules, we get:

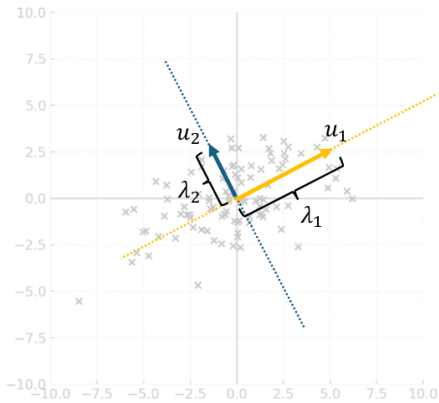
$$\nabla_u \mathcal{L} = 2\Sigma u - 2\lambda u = 0$$

If we divide the entire equation by 2 and rearrange it:

$$\boxed{\Sigma u = \lambda u}$$

In other words, any stationary point (our candidate for the optimum vector) must be an eigenvector of the covariance matrix Σ, and the multiplier λ corresponds to its eigenvalue.

4. **Decide which eigenvector gives the maximum**. Because our objective was to maximize $u^\top \Sigma u$, and we know that $\Sigma u = \lambda u \implies u^\top \Sigma u = \lambda$, the solution is simply the eigenvector with the largest eigenvalue. Such eigenvector is called **Principal Component 1 (PC1)**, denoted as u_1 with eigenvalue λ_1. We can then define u_2 as the eigenvector with the second-largest eigenvalue λ_2, and so on.



2.2.5 Variance explained

We would like to understand how much of our data's original information is actually preserved when we throw away some of the dimensions.

In PCA, information is directly equated to variance. Therefore, each principal component explains a specific proportion of the total variance within the data. If we were to keep all of our new components, between them, they would explain everything, i.e., they capture all the variability in the data. By design, PC1 always explains the most variance, PC2 explains the next highest amount, and so on.

To understand how much information a single component holds, we first need to know how much information exists in total. Assuming that all of our original variables have been centered to have zero mean, the total variance present in a dataset is simply the sum of the variances of the original variables:

$$Var_{TOT} = \sum_{j=1}^{p} Var(X_j) = \sum_{j=1}^{p} \left(\frac{1}{n} \sum_{i=1}^{n} x_{ij}^2 \right)$$

Projecting our data onto our new principal components, we get a new set of coordinates z (scores). The **variance explained** by any specific k-th principal component is calculated using these scores:

$$Var_k = \frac{1}{n} \sum_{i=1}^{n} z_{ik}^2$$

Therefore, to find out the percentage of the variance that is explained by our new system, we look at the **proportion of variance explained**. Assuming we have K components, it is defined as:

$$Var_{exp} = \frac{Var_K = \sum_{k=1}^{K} Var_k}{Var_{TOT}}$$

From Section 2.2.4, we have that the variance of the data along a principal component is perfectly equal to that component's eigenvalue:

$$Var_k = Var(z_{\cdot k}) = \frac{1}{n}\sum_{i=1}^{n} z_{ik}^2 = u_K^\top \Sigma u_k = u_k^\top(\lambda_k u_k) = \lambda_k$$

Now, we said that the total variance is simply the sum of the variances of all the original variables $(X_1, X_2, \dots)$. In the covariance matrix Σ this individual variances sit right on the diagonal. The sum of a matrix's diagonal is called **trace** (tr). Furthermore, we have that the trace of a matrix is always equal to the sum of all its eigenvalues. Therefore:

$$Var_{TOT} = \sum_{j=1}^{p} Var(X_j) = \mathrm{tr}(\Sigma) = \sum_{k=1}^{p} \lambda_k$$

Now that we know the variance of the individual components and the total variance, we can calculate the percentage of the variance that the k-th component explains:

$$\text{Explained variance ratio of } k\text{-th PC} \qquad EVR_k = \frac{\lambda_k}{\sum_{m=1}^{p} \lambda_m}$$

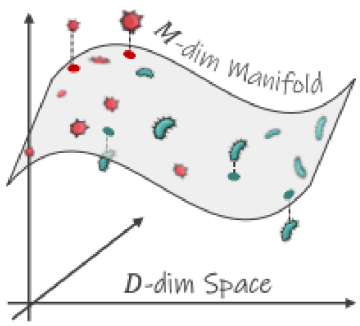
Since we want to reduce the number of dimensions (i.e., using less principal components, $K < p$), we want to know how much total information is kept:

$$\text{Cumulative explained variance } (K \text{ PCs}) \qquad \mathrm{CumEV}(K) = \frac{\sum_{k=1}^{K} \lambda_k}{\sum_{m=1}^{p} \lambda_m}$$

2.3 Non-linear dimensionality reduction

PCA has a fundamental limitation: it is a strictly linear dimensionality reduction technique. It assumes that the data's underlying structure can be captured by drawing straight lines (hyperplanes) through the space. Real-world data is rarely that simple. In order to reduce the dimensionality of complex real data, we need to find a way to properly model structures that are highly non-linear

To understand why linear techniques fail, we must understand how high-dimensional data actually behaves. This brings us to a concept known as the **Manifolds Assumption**: *"high-dimensional real data is not scattered uniformly across the ambient space. Rather, it often lies on a much lower dimensional, curved manifold."*.



In other words, even if a dataset is measured with thousands of variables (a massive ambient space), the actual meaningful variation in the data is usually constrained to a much smaller, highly curved, low-dimensional manifold embedded within that space. Therefore, in order to reduce dimensionality of real data, we need to find a way to properly model the non-linear, complex shape of the manifolds that best approximates them.

To tackle these complex curved manifolds we can rely on three main philosophies, each with a different mathematical objective:

1. **Linearize the structure (Kernel PCA family)**: the data might be highly non-linear in its original space $\mathbb{R}^D$, but if we apply a non-linear feature mapping function $\phi(x)$, the data might become "more linear" in a new, higher-dimensional space. Once it is linearized, we can then apply standard linear techniques.

2. **Preserve geometry via local similarity (t-SME / UMAP family)**: if two data points are near each other in the original high-dimensional space X, they must remain near each other in the new low-dimensional space Y. This is particularly suitable for visualizing complex clusters of data.

3. **Learn a non-linear mapping (Autoencoder family)**: learn an explicit non-linear function *encoder* f_θ, to compress the data into an embedding ($y_i = f_\theta(x_i)$), and a *decoder* g_ϕ, to reconstruct the original data ($\hat{x}_i = g_\phi(y_i)$). This is valuable as it provides a reusable non-linear embedding function *and* a (approximate) way to go back to the original space.